Arrays, Structures, and Abstract Data Types

Lecture 03

Data Structures and Algorithms

Pranshu Mandal

Part 1: Array Fundamentals

What is an Array?

An array stores elements of the **same type** in **contiguous memory locations**.

Key Properties:

- **Fixed size:** Number of elements decided at creation (static arrays)
- **Homogeneous:** All elements must be the same data type
- **Index-based access:** Direct access using index (position number)
- **Contiguous memory:** Elements sit next to each other — no gaps

```
int marks[5] = {85, 92, 78, 96, 88};
//  Index:       0   1   2   3   4
```

Analogy: A row of lockers in a hallway — each has a number, they're all the same size, and right next to each other. Know the number? Walk straight to it.

Memory Representation of 1D Arrays

When you declare `int arr[5]`, the computer reserves 5 consecutive blocks of memory (each `sizeof(int)` = 4 bytes).

Address Calculation Formula:

```
Address(arr[i]) = Base_Address + (i × Element_Size)
```

Example: `int arr[5]` starting at address 2000:

```
Index:     0       1       2       3       4
Value:    [85]    [92]    [78]    [96]    [88]
Address: 2000    2004    2008    2012    2016
```

- `arr[0]` → 2000 + (0 × 4) = **2000**
- `arr[2]` → 2000 + (2 × 4) = **2008**
- `arr[4]` → 2000 + (4 × 4) = **2016**

Why Address Calculation Matters

Because each element's address can be calculated with a **simple formula**, accessing *any* element takes the same time — **O(1)**.

The computer doesn't walk through the array to find an element; it **jumps directly** to the right address.

```
Want arr[3]?
→ 2000 + (3 × 4) = 2012
→ Go directly to address 2012
→ Done! No searching needed.
```

This is the key advantage of arrays:

Direct access in constant time, regardless of array size.

Part 2: 2D Arrays and Memory Layout

2D Arrays: Logical View

A 2D array is like a table with rows and columns:

```
Logical View (3×4 matrix):
         Col 0   Col 1   Col 2   Col 3
Row 0  [  10      20      30      40  ]
Row 1  [  50      60      70      80  ]
Row 2  [  90     100     110     120  ]
```

But computer memory is **linear** — a single long sequence of bytes.

The 2D structure must be "flattened" into a 1D sequence.

Row-Major Order

In C, 2D arrays are stored in **row-major order**: all elements of Row 0 come first, then Row 1, then Row 2...

```
Memory Layout (Row-Major):
[10][20][30][40][50][60][70][80][90][100][110][120]
|--- Row 0 ---||--- Row 1 ---||---- Row 2 ----|
```

Address Calculation Formula:

```
Address(arr[i][j]) = Base + (i × cols + j) × Element_Size
```

2D Address Calculation: Example

Given: `int arr[3][4]` starting at address 1000, `sizeof(int)` = 4

```
Address(arr[i][j]) = 1000 + (i × 4 + j) × 4
```

Calculate:

- `arr[0][0]` → 1000 + (0 × 4 + 0) × 4 = **1000**
- `arr[1][2]` → 1000 + (1 × 4 + 2) × 4 = 1000 + 24 = **1024**
- `arr[2][3]` → 1000 + (2 × 4 + 3) × 4 = 1000 + 44 = **1044**

Column-Major Order (For Reference)

Some languages (Fortran) use **column-major order** — columns stored first:

```
Column-Major:
[10][50][90][20][60][100][30][70][110][40][80][120]
|-- Col 0 --||-- Col 1 ---||- Col 2 --||-- Col 3 -|
```

Formula: `Address(arr[i][j]) = B + (j × rows + i) × S`

Why Memory Order Matters: Cache Locality

In C, always traverse 2D arrays row by row for best performance!

```
// FAST: Row-by-row (matches memory layout)
for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
        process(arr[i][j]);

// SLOW: Column-by-column (jumps around memory)
for (int j = 0; j < cols; j++)
    for (int i = 0; i < rows; i++)
        process(arr[i][j]);
```

Why? When you access `arr[0][0]`, the CPU loads nearby memory into cache. Row-by-row access uses that cache. Column-by-column access keeps fetching from distant locations — **cache misses** make it significantly slower.

Part 3: Array Operations

Insertion: At the End

Insert at the End (if space available):

- Simply place the element at `arr[size]` and increment `size`
- **Time: O(1)** — no shifting needed

```
Before: [10][20][30][40][ ]    Insert 50 at end
                                     ↓
Result: [10][20][30][40][50]   Just place it!
```

Insertion: At the Beginning

Every element must shift one position to the right.

Time: O(n) — must move all n elements.

```
void insert_at_beginning(int arr[], int *n, int value) {
    for (int i = *n; i > 0; i--) {
        arr[i] = arr[i - 1];  // Shift right
    }
    arr[0] = value;
    (*n)++;
}
```

Insertion: Visualization

```
Before: [10][20][30][40][ ]   Insert 99 at index 0
                                  ↓
Step 1: [10][20][30][ ][40]   Shift 40 right
Step 2: [10][20][ ][30][40]   Shift 30 right
Step 3: [10][ ][20][30][40]   Shift 20 right
Step 4: [ ][10][20][30][40]   Shift 10 right
Result: [99][10][20][30][40]  Place 99 at index 0
```

Insert at Position `k`:

- Shift elements from position `k` onward by one position right
- **Time:** O(n - k), worst case **O(n)**

Deletion

Removing an element requires shifting remaining elements to fill the gap.

Delete from the End: Simply decrement `size` → **O(1)**

Delete from the Beginning/Middle: Shift elements left → **O(n)**

```
void delete_at(int arr[], int *n, int index) {
    for (int i = index; i < *n - 1; i++) {
        arr[i] = arr[i + 1];  // Shift left
    }
    (*n)--;
}
```

Deletion: Visualization

```
Before: [10][20][30][40][50]   Delete at index 1 (value 20)
                                       ↓
Step 1: [10][30][30][40][50]   Copy 30 left
Step 2: [10][30][40][40][50]   Copy 40 left
Step 3: [10][30][40][50][50]   Copy 50 left
Result: [10][30][40][50][ ]    Decrement size
```

Searching: Linear Search — O(n)

Check each element one by one. Works on sorted **and** unsorted arrays.

```
int linear_search(int arr[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) {
            return i;  // Found at index i
        }
    }
    return -1;  // Not found
}
```

Best case: O(1) — element at first position

Worst case: O(n) — element at last position or not found

Searching: Binary Search — O(log n)

Works only on **sorted** arrays. Repeatedly divides search space in half.

```
int binary_search(int arr[], int n, int target) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == target)
            return mid;             // Found!
        else if (arr[mid] < target)
            low = mid + 1;          // Search right half
        else
            high = mid - 1;         // Search left half
    }
    return -1;  // Not found
}
```

Binary Search: Step-by-Step Example

Search for 70 in `[10, 20, 30, 40, 50, 60, 70, 80, 90]`

```
Step 1: low=0, high=8, mid=4 → arr[4]=50 < 70 → search right
Step 2: low=5, high=8, mid=6 → arr[6]=70 == 70 → FOUND at index 6!
```

Only 2 comparisons instead of 7 (linear search would need 7).

For an array of 1,000,000 elements:

- **Linear search:** up to 1,000,000 comparisons
- **Binary search:** at most 20 comparisons ($\log_2 1{,}000{,}000 \approx 20$)

Array Operations: Complexity Summary

Operation	Best Case	Worst Case
Access by index	O(1)	O(1)
Insert at end	O(1)	O(1)
Insert at beginning	O(n)	O(n)
Insert at position	O(1)	O(n)
Delete from end	O(1)	O(1)
Delete from beginning	O(n)	O(n)
Linear search	O(1)	O(n)
Binary search (sorted)	O(1)	O(log n)

Applications of Arrays

Common Use Cases:

- Storing collections of similar data (student records, sensor readings, pixel colors)
- Implementing other data structures (Stacks, Queues built on arrays)
- Matrix operations (graphics, physics, machine learning)
- Lookup tables and hash tables

Advantages:

- O(1) random access — fastest data access possible
- Cache-friendly — sequential memory layout is great for CPUs
- Simple to use and understand

Limitations:

- Fixed size — cannot grow/shrink (static arrays)
- Expensive insertion/deletion — shifting elements is costly
- Wasted memory — if you over-allocate

Part 4: Introduction to Structures

What are Structures?

Arrays store the *same* type of data. But what if you need *different* types together?

Example: A student record has a name (string), roll number (int), and GPA (float).

A **structure** (`struct`) groups **heterogeneous** (different-typed) data into one unit.

```
struct Student {
    char name[50];
    int roll_no;
    float gpa;
};
```

Using Structures

```
struct Student s1;
s1.roll_no = 101;
s1.gpa = 8.5;
strcpy(s1.name, "Ananya");

printf("Name: %s\n", s1.name);      // Ananya
printf("Roll: %d\n", s1.roll_no);    // 101
printf("GPA: %.1f\n", s1.gpa);       // 8.5
```

Memory Layout:

```
|--- name[50] ---|-- roll_no --|-- gpa --|
|   50 bytes     |   4 bytes   | 4 bytes |
```

Members of different types stored one after another (with possible padding for alignment).

Structure vs. Array

Feature	Array	Structure
Data types	Homogeneous (same)	Heterogeneous (different)
Access	By index (`arr[i]`)	By member name (`s.name`)
Use case	Collection of similar items	Grouping related but different data
Example	Marks of 50 students	One student's complete record

Key insight: They complement each other!

```
struct Student class[50];  // Array of 50 student records!
class[0].roll_no = 101;
class[0].gpa = 8.5;
```

Part 5: Advanced Structure Concepts

Nested Structures

A structure can contain another structure as a member — building complex data representations.

```
struct Address {
    char street[100];
    char city[50];
    int pin;
};

struct Employee {
    char name[50];
    int emp_id;
    struct Address addr;  // Nested structure
};
```

Accessing Nested Members

```c
struct Employee e1;
strcpy(e1.name, "Ravi");
e1.emp_id = 2001;
strcpy(e1.addr.city, "Mumbai");
e1.addr.pin = 400001;

printf("City: %s\n", e1.addr.city);  // Mumbai
```

Dot chaining: `e1.addr.city`

- First access `addr` member of `e1`
- Then access `city` member of that address

Function Pointers as Structure Members

A **function pointer** stores the address of a function, allowing indirect calls.

Why put them in structures? Create structures that "know how to do things" — early object-oriented thinking in C!

```c
typedef int (*Operation)(int, int);

int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

struct Calculator {
    Operation op;
    int a, b;
};
```

Function Pointers: Usage

```c
struct Calculator calc;
calc.op = add;
calc.a = 10;
calc.b = 5;
printf("Result: %d\n", calc.op(calc.a, calc.b));  // 15

calc.op = mul;  // Change the operation!
printf("Result: %d\n", calc.op(calc.a, calc.b));  // 50
```

The OOP Connection:

Grouping data + functions that operate on that data — exactly what **classes** do in C++ and Java. Structures with function pointers are C's way of achieving "methods" on data.

Self-Referential Structures

A structure that contains a **pointer to the same type of structure**.

```
struct Node {
    int data;
    struct Node* next;  // Pointer to another Node
};
```

This is one of the most important concepts in data structures!

Why a Pointer, Not the Struct Itself?

```
// THIS IS ILLEGAL — infinite size!
struct Node {
    int data;
    struct Node next;  // Error: struct contains itself
};
```

If `Node` contained a full `Node`, that inner `Node` would also contain a `Node`, and so on **infinitely**. The compiler would never know how much memory to allocate.

A **pointer** has a fixed, known size (4 or 8 bytes) — regardless of what it points to.

The Foundation of Linked Data Structures

Self-referential structures create chains — **linked lists!**

```
Node 1            Node 2            Node 3
+------+---+    +------+---+    +------+------+
| data |   •|-->| data |   •|-->| data | NULL |
|  10  |    |    |  20  |    |    |  30  |      |
+------+---+    +------+---+    +------+------+
```

Each node "points" to the next node, forming a chain.

We'll study linked lists in detail in upcoming lectures — but **self-referential structures are the fundamental building block**.

Part 6: Anonymous Unions

What are Unions?

A **union** is like a structure, but all members **share the same memory**. Only one member holds a value at any time.

```
union Data {
    int i;        // 4 bytes
    float f;      // 4 bytes
    char c;       // 1 byte
};
// sizeof(union Data) = 4 (size of the LARGEST member)
```

Structure vs. Union: Memory Layout

```
Structure:
|-- int i --|-- float f --|-- char c --|
|  4 bytes  |   4 bytes   |   1 byte   |  Total: 9+ bytes

Union:
|-------- shared memory --------|
|  4 bytes (any one value)      |  Total: 4 bytes
```

Structure: Each member gets its own space → more memory

Union: All members share the same space → saves memory but only one value at a time

Anonymous Unions Inside Structures

An **anonymous union** has no name — members accessed as if they belong to the enclosing structure.

```
struct Value {
    int type;  // 0 = int, 1 = float, 2 = string
    union {    // Anonymous union — no name!
        int int_val;
        float float_val;
        char str_val[20];
    };
};
```

Anonymous Unions: Usage

```c
struct Value v1;
v1.type = 0;
v1.int_val = 42;      // Access directly, no union name needed

struct Value v2;
v2.type = 1;
v2.float_val = 3.14;  // Only one union member used at a time

// Processing based on type
switch (v1.type) {
    case 0: printf("Int: %d\n", v1.int_val); break;
    case 1: printf("Float: %.2f\n", v1.float_val); break;
    case 2: printf("String: %s\n", v1.str_val); break;
}
```

Why? Memory efficiency — instead of space for int AND float AND string, the union shares it. With thousands of `Value` objects, savings add up!

Note: C11 feature — compile with `gcc -std=c11`.

Part 7: Abstract Data Types (ADTs)

What is an ADT?

An **Abstract Data Type** is defined by its **behavior** (what operations can be performed), not its **implementation** (how they work internally).

Vending machine analogy:

- **Interface (what you see):** Insert coin, press button, get item
- **Implementation (hidden inside):** Machinery, inventory, payment processing
- **You don't need to know the internals to use it!**

An ADT defines three things:

1. **Data:** What kind of data is stored
2. **Operations:** What actions can be performed
3. **Behavior Rules:** Rules governing operations (e.g., LIFO, FIFO)

ADT = Specification, Not Code

The key principle: **separation of interface from implementation** → **encapsulation**

Why is this powerful?

- Swap implementations without changing code that uses the ADT
- Multiple teams work independently: one designs interface, another implements
- Bugs in implementation don't affect the interface contract

ADT Example 1: List ADT

A List stores an ordered collection of elements.

Operation	Description
`insert(x, pos)`	Insert element x at position pos
`delete(pos)`	Remove element at position pos
`find(x)`	Find position of element x
`get(pos)`	Return element at position pos
`size()`	Return number of elements
`isEmpty()`	Check if list is empty

Possible implementations: Array, Linked List, Dynamic Array — the ADT doesn't care which!

ADT Example 2: Stack ADT — LIFO

Last In, First Out — like a stack of plates. Add and remove from the top only.

Operation	Description
`push(x)`	Add element x to the top
`pop()`	Remove and return the top element
`peek()`	Return the top element without removing
`isEmpty()`	Check if stack is empty

```
push(10) → push(20) → push(30) → pop()

  [30] ← top          [30] removed
  [20]                 [20] ← new top
  [10]                 [10]
```

Real-world uses: Undo (Ctrl+Z), browser back button, function call stack.

ADT Example 3: Queue ADT — FIFO

First In, First Out — like a line at a ticket counter. First person in is served first.

Operation	Description
`enqueue(x)`	Add element x to the rear
`dequeue()`	Remove and return the front element
`front()`	Return the front element without removing
`isEmpty()`	Check if queue is empty

```
enqueue(10) → enqueue(20) → enqueue(30) → dequeue()

Front → [10][20][30] ← Rear

After dequeue: 10 is removed
Front → [20][30] ← Rear
```

Real-world uses: Printer jobs, CPU scheduling, network message queues.

ADTs vs. Data Structures

This is a **crucial distinction** that ties together this entire course:

Aspect	ADT	Data Structure
Defines	Behavior (operations + rules)	Implementation (storage + processing)
Level	Abstract (specification)	Concrete (code)
Analogy	Blueprint / Contract	The actual building
Example	List ADT	Array or Linked List

ADT = What (specification)

Data Structure = How (implementation)

Same ADT, Multiple Implementations

```
List ADT
  ├── Array implementation
  │     → O(1) access, O(n) insertion
  └── Linked List implementation
        → O(n) access, O(1) insertion*

Stack ADT
  ├── Array implementation
  └── Linked List implementation

Queue ADT
  ├── Array (circular buffer) implementation
  └── Linked List implementation
```

The choice depends on which operations you use most frequently — and now you have the analysis tools from Lecture 02 to decide!

Part 8: Putting It All Together

The Journey So Far

Lecture	Topic	Key Takeaway
Lecture 1	Introduction to DS	Why data structures matter, memory concepts, problem-solving
Lecture 2	Algorithm Analysis	Tools to measure efficiency — Big-O, Big-Ω, Big-Θ, amortized
Lecture 3	Arrays, Structures, ADTs	First data structure, building blocks for complex types, ADT paradigm

How Everything Connects

```
Concepts (Lec 1)
     ↓
Analysis Tools (Lec 2)
     ↓
Basic Structure: Array (Lec 3)
     ↓
Building Blocks: Structs (Lec 3) → Self-referential structs → Linked Lists
     ↓
ADT Paradigm (Lec 3) → List, Stack, Queue ADTs → Multiple Implementations
```

Looking Ahead

Armed with these fundamentals, you're ready for:

- **Linked Lists:** Using self-referential structures to create dynamic, flexible storage
- **Stacks and Queues:** Implementing the ADTs we defined today
- **Trees and Graphs:** More complex structures built on the same principles
- **Algorithm Design:** Choosing the right data structure using analysis skills

Key Takeaways

What We Learned Today

1. Arrays

- Contiguous memory storage with O(1) index-based access
- 1D and 2D memory representation and address calculations
- Operations: insertion O(n), deletion O(n), linear search O(n), binary search O(log n)
- Advantages (fast access, cache-friendly) and limitations (fixed size, costly inserts)

2. Structures

- Group heterogeneous data into a single unit
- Nested structures for complex data models
- Function pointers as members — early OOP thinking
- Self-referential structures — building block for linked data structures

What We Learned Today (cont.)

3. Anonymous Unions

- Shared memory for different data types
- Memory-efficient variant types
- C11 feature

4. Abstract Data Types (ADTs)

- Separation of interface (what) from implementation (how)
- ADT = specification; Data Structure = implementation
- Same ADT → multiple possible implementations
- List, Stack (LIFO), Queue (FIFO) as foundational ADTs

Review Questions

1. What is the address formula for `arr[i]` in a 1D array?
2. Given `int mat[4][5]` at address 3000, what is `mat[2][3]`'s address (row-major)?
3. What is the time complexity of inserting at the beginning of an array?
4. What is a self-referential structure? Give an example.
5. Explain the difference between an ADT and a data structure with an example.
6. What is LIFO, and which ADT follows it?
7. Why can't a structure contain an instance of itself (without a pointer)?